

Analysis of Algorithms (AOA)

Assignment No. 1 & Assignment No. 2 - Complete Answer Book

Syllabus source check: PHCET's official Syllabus page lists **Computer - Second Year - Sem. III & IV** under the Autonomous B.Tech scheme and links the semester syllabus. The indexed syllabus copy identifies the Analysis of Algorithm course as CE202 and explicitly includes **Sum of subsets** under Backtracking and **Knuth-Morris-Pratt (KMP)** under String Matching.

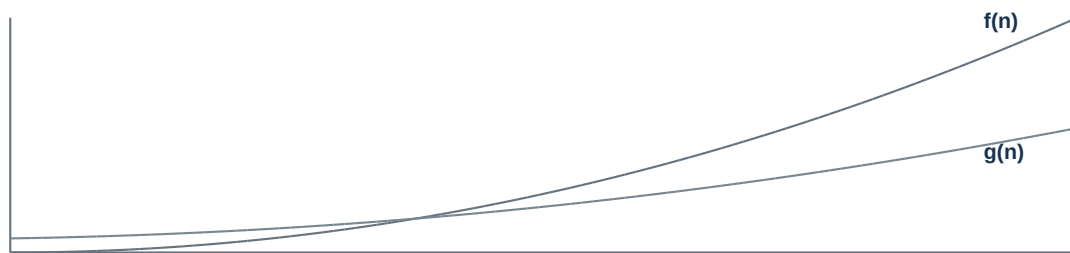
Source discipline: The first eight answers are based on the supplied five module notes. The final two answers are completed using the relevant topics in the PHCET Computer Engineering syllabus, as requested.

Assignment No. 1

1. Explain asymptotic notations.

- **Big-O, $O(g(n))$:** upper bound of growth; used for worst-case analysis.
- **Big-Omega, $\Omega(g(n))$:** lower bound of growth; used for best-case analysis.
- **Big-Theta, $\Theta(g(n))$:** tight asymptotic bound when upper and lower bounds have the same order.
- Algorithm analysis considers best, average and worst cases.

Asymptotic growth comparison



Definitions: $O(g(n))$ means $f(n) \leq c \cdot g(n)$ for sufficiently large n . $\Omega(g(n))$ means $f(n) \geq c \cdot g(n)$ for sufficiently large n . $\Theta(g(n))$ means $f(n)$ is bounded above and below by constant multiples of $g(n)$.

2. Write algorithm for Quick Sort and sort (40,11,4,72,17,2,49).

Algorithm:

```
QUICKSORT(A, low, high)
1. Select a pivot element.
2. Partition the array: smaller elements go to the left and greater elements to the right.
3. Recursively sort the left subarray.
4. Recursively sort the right subarray.
5. Stop for a subarray with zero or one element.
```

Sorting result: 11,4,17,2 are placed to the left of pivot 40 and 72,49 to the right. Recursive sorting gives 2,4,11,17 and 49,72.

Quick-sort partitioning example

```
[40, 11, 4, 72, 17, 2, 49]
      |
[11, 4, 17, 2] | 40 | [72, 49]
      |
[2, 4, 11, 17] | 40 | [49, 72]
      |
[2, 4, 11, 17, 40, 49, 72]
```

3. Write an algorithm for Binary Search and explain its working with an example.

Algorithm:

```
BINARY_SEARCH(A, key)
1. low ← 1, high ← n
2. while low ≤ high
3.   mid ← (low + high)/2
4.   if key = A[mid], return mid
5.   else if key < A[mid], high ← mid - 1
6.   else low ← mid + 1
7. return -1 if key is absent
```

For 10,20,30,40,50,60,70 and key 60: mid = 40, so search the right sublist; then mid = 60 and the key is found.

Binary-search example: key = 60



mid = 40 -> 60 > 40 -> search right sublist

mid = 60 -> key found

Time complexity: Best $O(1)$, Average $O(\log n)$, Worst $O(\log n)$.

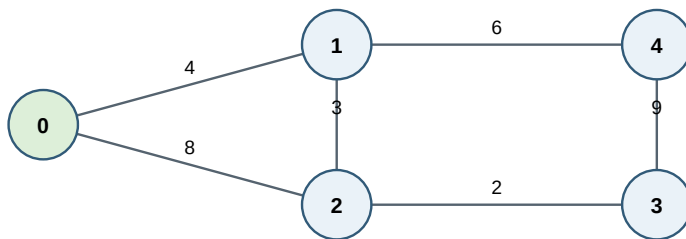
4. Explain Dijkstra's algorithm with an example.

Definition / idea: Dijkstra's algorithm finds shortest paths from a specified source. The source is assigned cost 0; neighbouring path costs are updated, and the unvisited node with minimum current cost is selected repeatedly.

Algorithm:

1. Accept graph $G(V,E)$ and source node.
2. Mark source visited and set cost to 0.
3. Transfer/update path costs to direct neighbours; select the unvisited neighbour with minimum cost.
4. Continue until the required nodes are visited.
5. Print the solution.

Dijkstra example (source = 0)



Distances: 0, 4, 7, 9, 10

Example: From source 0, the shortest distances are $d(0)=0$, $d(1)=4$, $d(2)=7$, $d(3)=9$ and $d(4)=10$.

5. Analyse the Greedy Knapsack algorithm and obtain the solution.

Given: Capacity $M=8$; weights (2,3,4,5); profits (3,5,6,10).

Item	Weight	Profit	P/W
W1	2	3	1.50
W2	3	5	1.67
W3	4	6	1.50
W4	5	10	2.00

Descending P/W order: W4, W2, W1, W3. Choose W4 (weight 5, profit 10), leaving capacity 3; choose W2 (weight 3, profit 5).

Solution: $x=(0,1,0,1)$, total weight=8, total profit=**15**.

Assignment No. 2

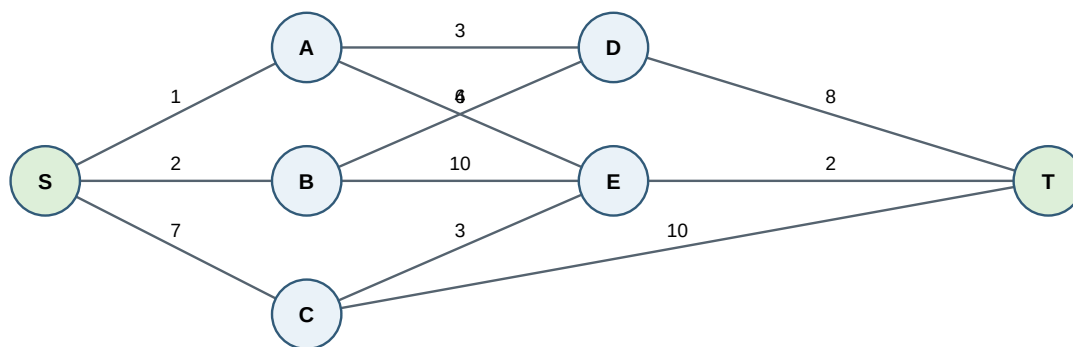
1. Explain Multistage Graph with an example.

A multistage graph is a directed weighted graph divided into stages. The aim is to find the minimum-cost path from the starting stage to the final stage. The supplied Dynamic Programming notes solve it by calculating minimum costs backwards from the destination.

Algorithm:

1. Accept a directed weighted graph.
2. Identify stages and source/destination.
3. Starting at the last stage, calculate minimum cost to the destination from each node.
4. Continue backwards to the starting node.
5. Select the minimum-cost path.

Multistage graph example



Optimal path: S → A → E → T ; cost = 9

Result for the shown example: $d(D,T)=8$, $d(E,T)=2$, $d(A,T)=8$, $d(B,T)=12$, $d(C,T)=5$ and $d(S,T)=9$. Hence $S \rightarrow A \rightarrow E \rightarrow T$ is optimal.

2. Find the Longest Common Subsequence for X=ACBAED and Y=ABCABE.

Use the Dynamic Programming table $c[i,j]$. If $x_i=y_j$, take $c[i-1,j-1]+1$; otherwise take $\max(c[i-1,j], c[i,j-1])$.

	A	B	C	A	B	E	
A	0	1	1	1	1	1	1
C	0	1	1	2	2	2	2
B	0	1	2	2	2	3	3
A	0	1	2	2	3	3	3
E	0	1	2	2	3	3	4
D	0	1	2	2	3	3	4

Backtracking: Following the selected entries from the bottom-right cell gives the LCS **ACBE**. Length = 4.

3. Explain N-Queens Problem with an example.

The N-Queen problem places queens on an $n \times n$ board so that no two queens attack each other in the same row, column or diagonal. It is solved using backtracking.

Algorithm:

1. Place a queen in the first column of the first row.
2. Check conflict with previously placed queens.
3. If safe, proceed to the next column; otherwise backtrack and try the next row.
4. Repeat for all columns.
5. When all queens are placed safely, a solution is found.
6. If no position works, backtrack to the previous column.

4-Queen

	Q		
			Q
Q			
		Q	

Q1: row 1, col 2

Q2: row 2, col 4

Q3: row 3, col 1

Q4: row 4, col 3

For the 4×4 example, the solution vector given in Module 5 is **(2,4,1,3)**.

4. Explain Subset Sum Problem using Backtracking (N=6, W={3,5,7,8,9,15}, M=20).

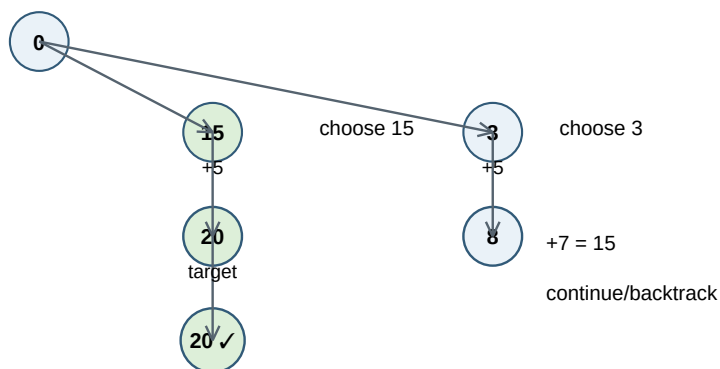
The PHCET Computer Engineering syllabus places **Sum of subsets** in Unit V under Backtracking. The problem asks us to find subsets whose sum is exactly the target M=20. Backtracking explores the include/exclude choice for each weight and abandons a branch when the partial sum cannot lead to 20.

Backtracking algorithm:

```
SUM_OF_SUBSETS(k, currentSum, remainingSum)
```

1. Include the next element and update currentSum.
2. If currentSum = M, report the subset.
3. If currentSum > M, backtrack.
4. Otherwise recursively continue with the next element.
5. Also explore the branch that excludes the current element when it can still lead to a solution.
6. Continue until all elements have been considered.

Subset-sum backtracking (target M = 20)



Solutions: {15,5} and {3,8,9}

Given set: {3,5,7,8,9,15}. Target = 20. Valid subsets are:

- 1) {15,5} → 15+5=20
- 2) {3,8,9} → 3+8+9=20.

Final answer: There are **two solutions**: {5,15} and {3,8,9}.

5. Give algorithm for KMP string matching algorithm.

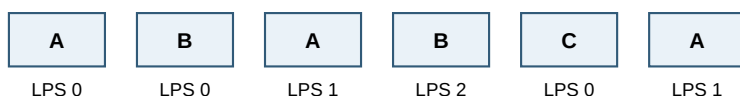
The PHCET Computer Engineering syllabus lists **Knuth-Morris-Pratt (KMP)** under Unit VI: String Matching Algorithms. KMP uses prefix/suffix information from the pattern so that a mismatch does not force the text index to move backwards.

KMP algorithm:

KMP_MATCH(T, P)

1. Construct the LPS (Longest Proper Prefix which is also Suffix) array for pattern P.
2. Set $i=0$ for text T and $j=0$ for pattern P.
3. If $T[i]=P[j]$, increment i and j .
4. If j reaches the pattern length, a match is found at $i-j$; set $j=LPS[j-1]$ to continue searching.
5. If a mismatch occurs and $j>0$, set $j=LPS[j-1]$ without changing i .
6. If a mismatch occurs and $j=0$, increment i .
7. Stop when i reaches the end of the text.

KMP idea: use prefix/suffix (LPS) information to avoid rechecking



On mismatch: $j = LPS[j-1]$; on match: $i++$, $j++$.

The text index i is not moved backwards.

LPS construction: For each pattern position, $LPS[j]$ stores the length of the longest proper prefix of the pattern that is also a suffix ending at j . This table is what allows KMP to skip unnecessary comparisons.

Time complexity: LPS preprocessing $O(m)$ and matching $O(n)$, so total $O(n+m)$.

Sources used

1. PHCET official Syllabus page - Bachelor of Technology, Computer, Second Year, Sem. III & IV (Autonomous).
2. PHCET / University syllabus content for Analysis of Algorithm (course CE202), including Backtracking - Sum of subsets and String Matching - Knuth-Morris-Pratt (KMP).
3. User-provided AOA module PDFs for the remaining assignment answers.